

Storage Wars

Redis does one thing at a time. That's why it's fast.

Issue #3

Sat, 03 Oct 2026

🕒 ~21 min read

Level 3/5

⚡ The short version

1. **Redis runs your commands on one thread, and that's a feature.** Everything lives in RAM, one event loop serves every client, and small values get compact encodings. Part 1 covers the data structures, the event loop, and the one slow command that blocks everybody.
2. **Databricks changed what "delete pipeline" means (25 Sep):** deleting a Unity Catalog pipeline now **keeps** its tables by default. Check your cleanup scripts.
3. **Two big key-value case studies:** Perplexity replaced DynamoDB with **CobbleDB**, its own Rust store on RocksDB, and Meta put the **ZGateway** proxy in front of ZippyDB to stop connection storms from more than a million clients.

Release radar

8.10

latest **Redis Open Source** line listed in the redis.io configuration docs

18.6

current **PostgreSQL** minor; **19** is still at **Beta 4** (24 Sep), with no RC yet

40 days

until **PostgreSQL 14** end-of-life on 12 Nov 2026

Sources: Redis configuration docs, postgresql.org, PostgreSQL versioning policy.

PART 1 Learn

⚡ Redis I: data structures, one thread, and why it's fast

SYSTEMS

~10 MIN

LEVEL 3

Your Postgres is running dozens of backend processes, a buffer pool and a WAL writer. Redis runs your commands on one thread, and it's still usually the fastest thing in your stack. That sounds backwards until you see where the time actually goes. In the last two issues we followed a row through pages on disk and an index through a B-tree. Redis skips nearly all of that: **the data is in RAM, the structures are built for the commands, and nothing waits on a lock.**

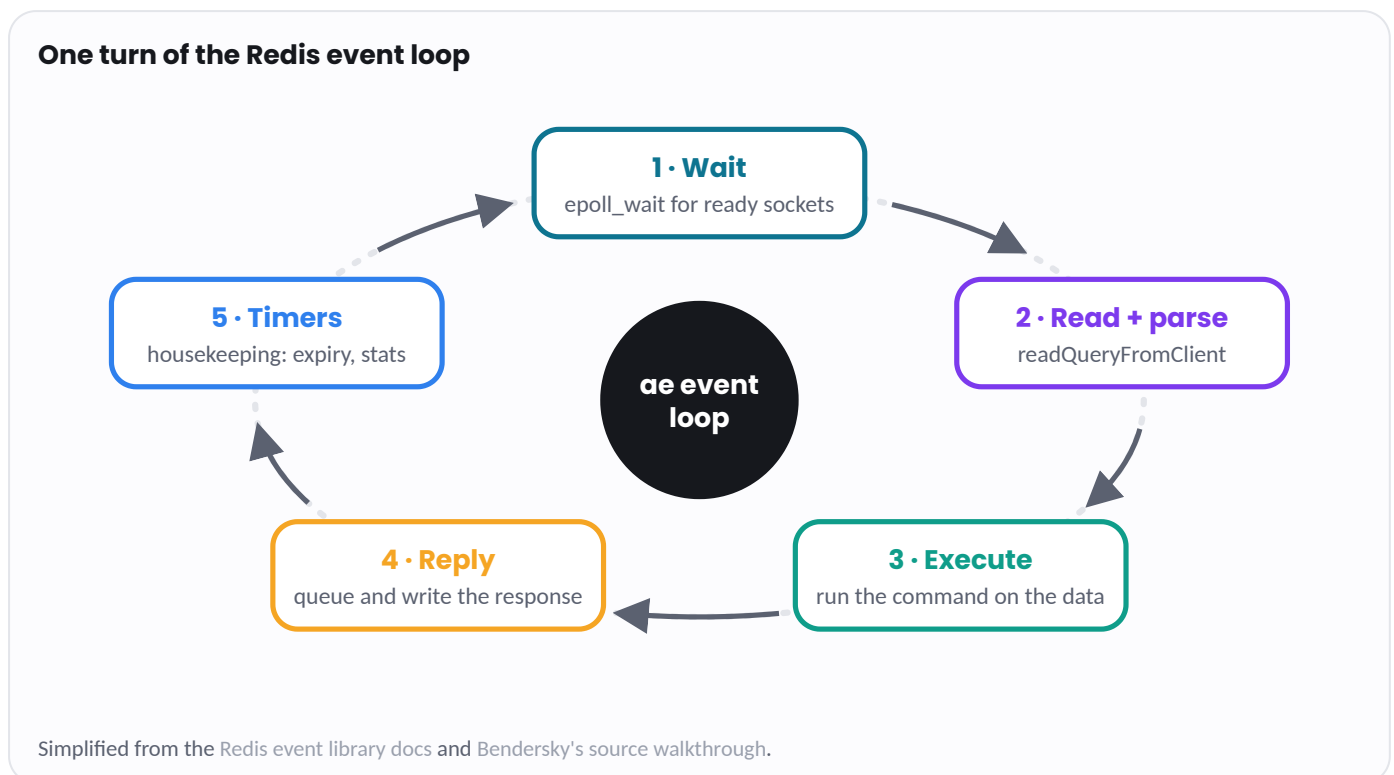
FOUR REASONS REDIS IS FAST

1. **Everything is in memory.** There's no buffer pool deciding what's hot and no page to fetch from disk. Every read touches RAM.
2. **One thread runs the commands.** Redis's own course puts it this way: single-threading **avoids race conditions and CPU-heavy context switching**, and **CPU is rarely the bottleneck**; you usually run out of memory or network first. (Redis University)

3. **Purpose-built encodings.** Small hashes, sets and sorted sets are stored as compact byte arrays instead of pointer-heavy hash tables. That saves memory and keeps data close together in CPU cache.
4. **Non-blocking I/O.** One event loop watches every client socket at once, so a slow client never holds up a fast one.

THE EVENT LOOP: ONE THREAD, MANY CLIENTS

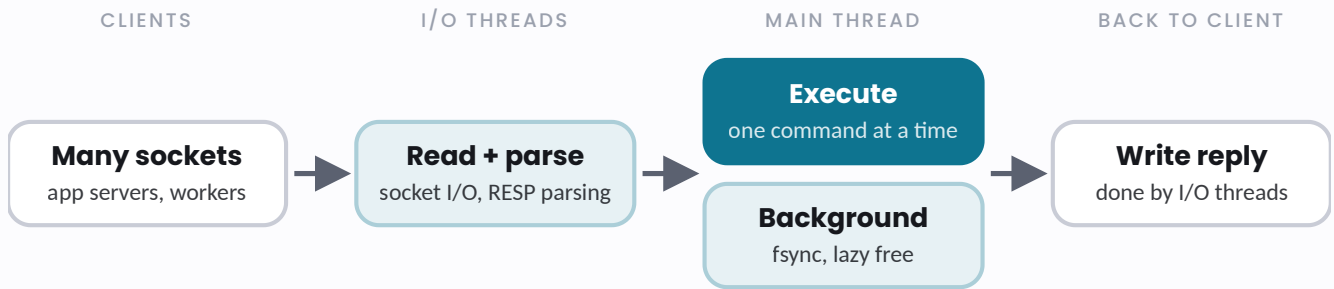
Redis doesn't use a library like libevent. It has its own small event library, `ae`, which wraps the operating system's readiness API (`epoll` on Linux). The loop registers sockets with the OS, waits until some are ready, then runs the callbacks for those: accept a connection, read a query, write a reply. Timers for periodic housekeeping run on the same loop. (Redis docs: event library) Eli Bendersky's walkthrough of the source sums up the bet: Redis is **network-bound, spending much more time waiting on the network than inside its data structures**, so non-blocking I/O on one thread is enough. (Eli Bendersky)



The pay-off is that **every command is atomic for free**. `INCR`, `LPUSH`, `ZADD`: no other command can run halfway through yours, because there is no other command running. You get this guarantee without any locks.

"Single-threaded" isn't quite true any more. Redis has used background threads for disk work since 2.4, for example the `fsync` behind `appendfsync everysec`. (Redis docs: latency) Redis 6.0 added **I/O threads** for socket reads and writes, which Redis says boosts throughput by **up to 2× for some workloads**. (Redis University) Redis 8.0 replaced them with a new asynchronous implementation, still enabled with `io-threads`. (Redis 8.0 release notes) The key detail: **I/O threads read, parse and write, but the main thread still executes every command.**

Redis 8 with io-threads: who does what



Based on the Redis 8.0-M03 announcement: the default is `io-threads 1`; Redis reported 37%–112% higher throughput with 8 I/O threads, depending on the command (vendor benchmark).

THE DATA STRUCTURES AND THEIR SECRET SECOND FORM

Every Redis type has **two encodings**: a **compact one for small values** and a **general one for big values**. Redis switches automatically when a value outgrows the compact form. You can see which one you have with

`OBJECT ENCODING key`. (Redis docs)

Type	Small encoding	Large encoding	Switches when (defaults, Redis ≥ 7)
String	<code>int</code> / <code>embstr</code>	<code>raw</code>	<code>embstr</code> holds up to 44 bytes
Hash	<code>listpack</code>	<code>hashtable</code>	> 512 fields or a value > 64 bytes
Sorted set	<code>listpack</code>	<code>skiplist</code> (+ hash table)	> 128 members or a member > 64 bytes
Set	<code>intset</code> / <code>listpack</code>	<code>hashtable</code>	> 512 integers; > 128 strings (7.2+)
List	<code>listpack</code> nodes	<code>quicklist</code>	a linked list of listpacks
Stream	n/a	<code>stream</code>	a radix tree of listpacks

Thresholds from the [memory optimization docs](#) and `OBJECT ENCODING`. They are tunable with `hash-max-listpack-entries` and similar settings.

A **listpack** stores entries back to back in one block of memory. Finding a field means scanning it, which is $O(N)$, but N is capped at a few hundred and the scan is cache-friendly. Redis says these special encodings use **up to 10× less memory (5× on average)**. (Redis docs)

Sorted sets deserve a special mention: the large form is a **skip list plus a hash table**. The skip list keeps members ordered by score for `ZRANGE` and rank queries, and the hash table answers `ZSCORE` in $O(1)$. It's the reason leaderboards and rate limiters are a natural fit for Redis.

Same hash, two shapes

listpack (small)

- One contiguous block, entries side by side
- Lookups scan, but N is small and cache-hot
- Far less memory: no pointers, no per-entry objects

```
HSET user:1 name Adi city Pune
```

VS

hashtable (large)

- Buckets of pointers to separately allocated entries
- O(1) lookups at any size
- Converted once the limits are crossed; it doesn't convert back on its own

```
513th field → OBJECT ENCODING =  
hashtable
```

Encodings and thresholds from the Redis docs.

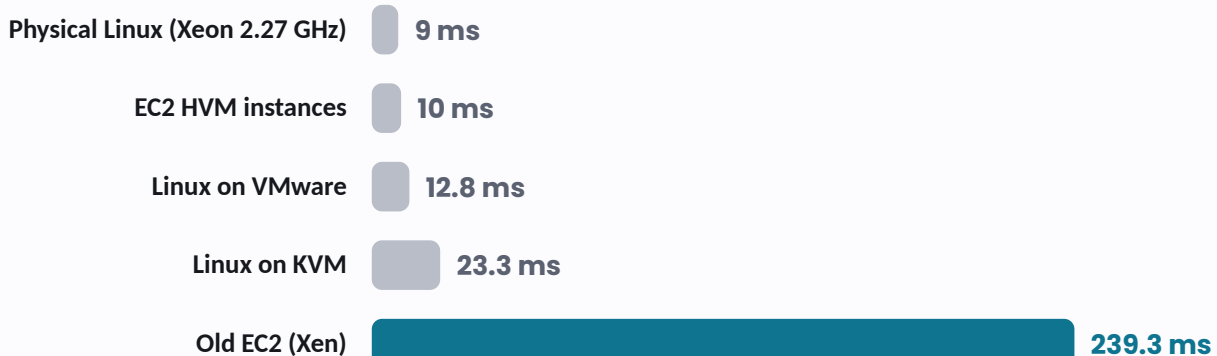
THE PRICE OF ONE THREAD: ONE SLOW COMMAND BLOCKS EVERYONE

Since commands run one after another, **the latency of the slowest command is added to every command queued behind it**. Redis's latency guide is blunt: `GET`, `SET` and `LPUSH` are fine, but O(N) commands such as `SORT`, `LREM` and `SUNION` on big values hurt, and **KEYS should only be used for debugging**. Use `SCAN`, `HSCAN`, `SSCAN` and `ZSCAN` to iterate in small steps, and watch `SLOWLOG`. (Redis docs: latency)

Deleting is a hidden case. `DEL` on a key with millions of elements frees every allocation on the main thread. **UNLINK removes the key from the keyspace in O(1) and frees the memory in another thread.** (Redis docs)

The other classic stall is `fork()`. RDB snapshots and AOF rewrites fork the process, and copying the page tables blocks the main thread for a time that grows with the dataset size:

`fork()` pause per GB of Redis memory



Measurements published in the Redis latency guide; older hardware, so treat them as orders of magnitude. Multiply by your dataset size: a 24 GB instance on a slow hypervisor can pause for seconds.

The same guide says to **disable transparent huge pages**, which make latency spikes after a fork much worse. (Redis docs: latency)

In the real world: Instagram's 300 million keys

Instagram needed to map about **300 million photo IDs to the user IDs** that owned them. Stored as plain string keys, a test of 1 million keys took about **70 MB**, which projected to roughly **21 GB** for the full set. The fix used the compact encoding from today's table: split each media ID into a **bucket** (the ID divided by 1,000) and store the pairs as fields in a hash per bucket, with the hash encoding limit tuned to about 1,000 entries. One million keys then took about **16 MB**, and the whole dataset fit in **just under 5 GB**.

(Instagram Engineering) The Redis docs describe the same trick and measured **11 MB → 1.7 MB** for 100,000 objects. (Redis docs)

Gotcha: the cache that never evicts

On 64-bit systems **maxmemory defaults to 0, meaning no limit**, so a "cache" grows until the OS kills it.

(Redis docs: eviction) Setting a limit isn't enough on its own: the **volatile-* policies only evict keys that have a TTL, and behave like noeviction if none do**. Then writes start failing with OOM errors while reads keep working. Managed services pick their own defaults; Google's Memorystore for Redis, for example, defaults to **volatile-lru**. (Google Cloud) Check with `CONFIG GET maxmemory-policy`. For a pure cache, the Redis docs call **allkeys-lru a good default**. Also leave headroom: replication and AOF buffers **don't count toward maxmemory**. (Redis docs: eviction)

Try it (15 minutes, Docker)

```
docker run -d --name rd redis:8
# 1. Watch a hash change shape at field 513
docker exec -i rd sh -c 'for i in $(seq 1 512); do
  echo "HSET user:1 f$i $i"; done | redis-cli > /dev/null'
docker exec -it rd redis-cli OBJECT ENCODING user:1 # listpack
docker exec -it rd redis-cli MEMORY USAGE user:1
docker exec -it rd redis-cli HSET user:1 f513 513
docker exec -it rd redis-cli OBJECT ENCODING user:1 # hashtable
docker exec -it rd redis-cli MEMORY USAGE user:1
# 2. Same server, with and without pipelining (-P 16)
docker exec -it rd redis-benchmark -t set,get -n 100000 -q
docker exec -it rd redis-benchmark -t set,get -n 100000 -q -P 16
# 3. What did that cost? Slow commands and the intrinsic floor
docker exec -it rd redis-cli SLOWLOG GET 5
docker exec -it rd redis-cli --intrinsic-latency 10
```

Compare memory before and after the 513th field, then the requests per second with and without `-P 16`. Pipelining sends many commands per round trip, so Redis does **one read() and one write() system call for many commands** instead of one each. The docs credit it with **up to 10×** higher throughput. (Redis docs: pipelining)



USE IT TOMORROW

1. **Never run `KEYS` in production.** Use `SCAN` and its type-specific variants.
2. Use `UNLINK` instead of `DEL` for anything that might be big.
3. **Keep hashes and sorted sets under the listpack limits** when you have millions of small objects. Bucket them if needed.
4. Set `maxmemory` and a policy that matches your keys, and check what your managed service picked for you.
5. **Pipeline batches** of commands, in chunks (the docs suggest about 10k), because the server holds the replies in memory until you read them.



Under the hood

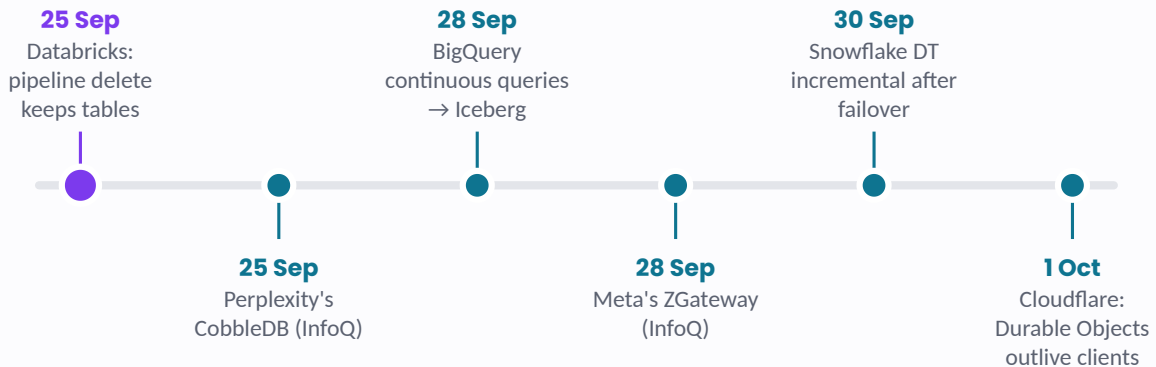
- **Expiry is lazy plus sampled.** On top of expiring keys when they're accessed, Redis runs an active cycle **10 times a second**, sampling **20 keys** with a TTL and deleting the expired ones. **If more than 25% of a sample has expired, it repeats.** A huge batch of keys expiring in the same second can therefore block the server. Add jitter to your TTLs. (Redis docs: latency)
- **LRU is approximate.** Redis samples `maxmemory-samples` (**5**) keys and evicts the oldest, keeping a pool of good candidates since 3.0. Raising it to 10 gets very close to true LRU, at some CPU cost. **LFU** uses a probabilistic **Morris counter** (0–255) that decays every minute by default. (Redis docs: eviction)
- **New in 8.6: LRM eviction.** `allkeys-lrm` updates the timestamp only on writes, so it evicts data that hasn't **changed** recently, even if it's read a lot. (Redis docs: eviction)
- **Find your floor.** `redis-cli --intrinsic-latency 100` measures what the OS and hypervisor add before Redis does anything. The docs cite about **0.115 ms** on a physical server versus up to **40 ms** in a bad virtualized environment. (Redis docs: latency)

GO DEEPER

- **Diagnosing latency issues** (Redis docs). The single best page on how the one-thread model breaks, from `fork` to expiry storms.
- **Memory optimization** and **Key eviction** (Redis docs). Encodings, thresholds, bucketing and every eviction policy.
- **Concurrent Servers, Part 5: Redis case study** (Eli Bendersky). A guided tour of the event loop in the actual C source.
- **Storing hundreds of millions of simple key-value pairs in Redis** (Instagram Engineering). Short, practical, and still a good lesson.

~10 min • A Databricks default flips, two giants build their own key-value plumbing, and Iceberg keeps spreading.

The week in data, 25 Sep → 2 Oct



Top story 1 • Databricks: deleting a pipeline no longer deletes its tables

BEHAVIOUR CHANGE

DATABRICKS

What happened. On **25 September**, Databricks changed the default: when you delete a **Unity Catalog pipeline** that uses the default publishing mode, **the UI and the Pipelines DELETE API now keep its materialized views, streaming tables and views**. To delete them along with the pipeline, set **cascade** to **true** in the API request. (Databricks release notes)

The same week brought two more governance changes. **Query tags for SQL warehouses** went GA on **30 September**: custom key-value tags on SQL workloads that show up in `system.query.history`, for grouping, filtering and **cost attribution**. And **ABAC row filters and column masks now apply to views** (Beta, Databricks Runtime 19+). (Databricks release notes)

Why it matters. It's the safer default, since nobody loses a table by deleting the wrong pipeline. But **automation that relied on cascade-by-default will now leave orphaned tables behind**, still costing storage and still visible to anyone with grants on the schema.

👁️ What to do. Search your Terraform, CI and cleanup scripts for pipeline deletes. Where tables should go with the pipeline, add `cascade: true` explicitly. If you pay one Databricks bill across several teams, start tagging warehouse queries now so next month's cost report can be split by team.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Production risk



Top story 2 · Perplexity swaps DynamoDB for its own Rust key-value store

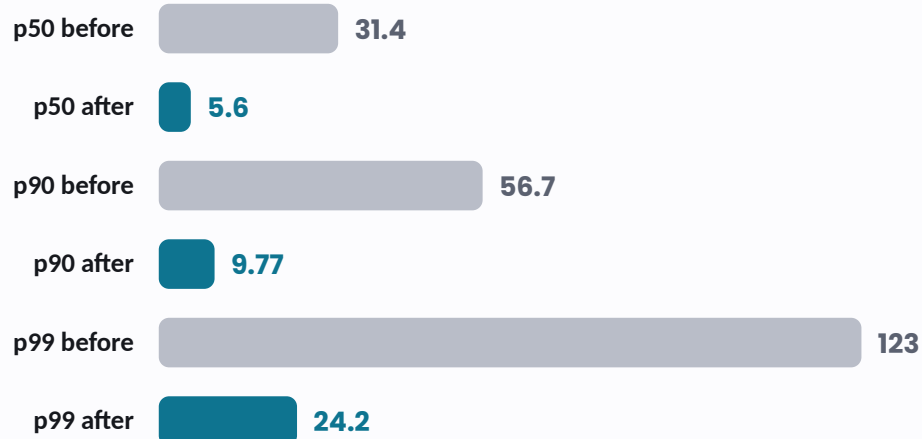
CASE STUDY

KEY-VALUE

What happened. InfoQ reported on **25 September** that Perplexity built **CobbleDB**, a distributed key-value store in Rust, to serve batched document lookups for its answer engine in place of DynamoDB. Perplexity gave three reasons. Usage-based pricing became unsustainable at **200,000+ requests per second**, since every byte transferred is billed. Tail latency was a black box it couldn't tune. And bulk reprocessing jobs competed with live user reads. (InfoQ)

How it's built. RocksDB with memory-mapped caching on local **NVMe** disks, **three replicas per partition** with **asynchronous replication**, which means eventual consistency. A stateless router handles zone affinity and **hedged requests**, and a batched `MultiGet` API cuts round trips. Writes arrive as batches from S3, so heavy ingestion is kept away from serving. It was about **40,000 lines of Rust** written in **two months** by two engineers plus AI coding agents, and Perplexity says it plans to open-source it. (InfoQ)

Batch read latency: DynamoDB → CobbleDB (ms)



Perplexity's own production numbers as reported by InfoQ; vendor claims, not independently verified. Perplexity also claims storage costs fell by at least 20%.

Why it matters. It's the same physics as Part 1: **keep hot data close to the CPU, batch round trips, and own your tail latency.** It's also a trade-off people tend to forget: Perplexity accepted **eventual consistency** to get there. For most teams, a managed store is still the right call. The lesson is to know **which line on the bill grows with your payload size.**

👁️ Should you care? Only if a managed key-value store is a top-three line on your bill. If it is, check whether you're paying per request, per byte or per capacity unit, and whether batching reads (`BatchGetItem` , `MGET` , `MultiGet`) would cut it before you think about building anything.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Top story 3 · Meta puts a proxy in front of ZippyDB to stop connection storms

CASE STUDY

ARCHITECTURE

What happened. Meta described **ZGateway**, a stateless proxy between clients and **ZippyDB**, its distributed key-value store for metadata, counters and configuration. More than **one million client hosts** connecting directly created a many-to-many mesh, and **connection storms** caused file-descriptor exhaustion and out-of-memory errors on database servers. ZGateway now handles **over 1 billion operations per second**, about **40% of ZippyDB traffic**, and cuts **per-host connections by 97–98%** (about **19× fewer** persistent connections). It also batches and coalesces requests, enforces **per-tenant admission control**, and sheds load under overload. (InfoQ), from [Meta's engineering post](#)

Why it matters. At scale, **connections become a resource you have to manage on their own**, separate from queries. Each connection costs memory and a file descriptor on the server, even for a single-threaded engine like Redis. It's the same problem PgBouncer solves for Postgres (a later Learn topic) and that Redis proxies solve for big clusters. **If your client count grows with your pod count, you'll hit this long before you hit CPU limits.**

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



⚡ Quick hits

📦 **BigQuery continuous queries write to Iceberg (28 Sep).**

Continuous queries can now write output rows directly into **Apache Iceberg managed tables**. The **Rust SDK for BigQuery** went GA on 2 Oct. (BigQuery release notes)

❄️ **Snowflake dynamic tables survive failover (GA, 30 Sep).**

INCREMENTAL and ADAPTIVE dynamic tables keep refreshing incrementally after failover, as long as the table and its base objects are in the same failover group and unchanged. Otherwise you get a full reinit; check `reinit_reason`. (Snowflake docs)

🌀 **AlloyDB caches vector indexes in its columnar engine (GA, 21 Sep).**

It now acts as a read-optimized, in-memory cache for **HNSW** indexes to raise vector-search QPS. Google gave no numbers. (AlloyDB release notes)

🔍 **Cosmos DB courts Elasticsearch users.**

Three posts in three days (30 Sep–2 Oct) on migrating mappings and translating search requests, plus **Cosmos DB Shell** inside the Azure portal (1 Oct). (Azure Cosmos DB blog)

☁️ **Cloudflare: Durable Objects outlive their clients (1 Oct).**

Each pending I/O operation now prevents idle shutdown for **up to 15 minutes**. **Basin Pipelines** streams also went from 5 MB/s to **1 GB/s**. (Cloudflare changelog)

🦆 **DuckDB: group on integers, not strings (2 Oct).**

Swap repeated strings for compact integer keys in a dimension table, aggregate on the keys, and join the names back at the end. No benchmark numbers; the post says the gain grows with row counts and string length. (DuckDB)

Databricks JDBC 2.8.4 supports Java 25 (29 Sep).

The Simba-based driver also upgrades its bundled libthrift, httpclient5, httpcore5 and lz4-java libraries, worth knowing if you shade or pin those yourself.

(Databricks release notes)

Coming up.

PostgreSQL 19 RC (still Beta 4 as of today), **DuckDB 2.0** (projected for the second half of October, [per DuckDB](#)), Postgres 14 end-of-life on 12 Nov, and MongoDB's permanent CEO (Dev Ittycheria is still interim).

Revision card

- Redis is fast because data is **in RAM**, commands run on **one thread** with no locks, and small values use **compact encodings**.
- The **ae event loop** uses **epoll** to serve many clients. Every command is **atomic** because nothing else runs at the same time.
- **I/O threads** (6.0; rebuilt in 8.0) read, parse and write. The **main thread still executes** every command.
- Small hashes/zsets/sets are **listpacks**; past **512** hash fields or **64**-byte values they become hash tables. Check with **OBJECT ENCODING**.
- One slow command blocks all clients: **no KEYS**, use **SCAN**; **UNLINK** instead of **DEL**; watch **SLOWLOG**.
- **fork()** **pauses grow with memory size**; disable transparent huge pages.
- **maxmemory 0 = unlimited** on 64-bit; **volatile-*** with no TTLs **acts like noeviction**.
- **Databricks pipeline delete now keeps tables**: add **cascade: true** where you mean it.

Storage Wars is researched and written by Claude every morning from official docs and release notes, project blogs and engineering write-ups. Vendor numbers are labelled as claims, and every item links to its source. Want more Kafka, less Mongo, or a different depth? Change `data/settings.yml` or just reply. 